Université Abderrahmane Mira de Béjaia
Faculté des Sciences Exactes
Département : Informatique

Mini Projet

Module : Génie Logiciel
Niveau : 3ème année ING

Analyse et Modélisation : Application de Gestion des Vols pour une Compagnie Aérienne

Chargée de cours : L. Hamza
Chargées TP : N. Bair & S. Boussoufa

Réalisé par :
BOUKHENAK Kamel
GUELAACHE HOUCINE

Remise du Rapport : 27 novembre 2025

Table des matières

1. Introduction 3
    1.1. Énoncé du projet 3
    1.2. Objectifs du mini-projet 3
2. Cahier des charges 4
    2.1. Présentation générale 4
    2.2. Périmètre du système 4
    2.3. Exigences fonctionnelles 4
    2.4. Contraintes fonctionnelles 5
    2.5. Exigences non fonctionnelles 5
    2.6. Modèle conceptuel 5
    2.7. Livrables 5
    2.8. Technologies recommandées 6
3. Fonctionnalités du système 6
    3.1. Gestion des avions 6
    3.2. Gestion des pilotes 6
    3.3. Gestion des vols 6
    3.4. Fonctionnalités de recherche 6
    3.5. Fonctionnalités générales du système 7
    3.6. Contraintes fonctionnelles 7
3. Diagrammes UML 7
    3.1 Diagramme de cas d'utilisation 7
    3.2 Diagramme de classes 9
    4.3 Diagrammes de séquence 10
5. Modèle de données (SQL) 14
    5.1 Script SQL 14
6. Modèle relationnel simplifié 16
    6.1. Tables principales 16
    6.2. Relations entre les tables 16
    6.3. Diagramme relationnel simplifié (textuel) 16
7. Fonctionnement détaillé de l'application 17
    7.1. Authentification et accès au système 17
    7.2. Gestion des avions 17
    7.3. Gestion des pilotes 17
    7.4. Gestion des vols 17
    7.5. Recherche et filtrage avancés 18
    7.6. Gestion des erreurs et intégrité des données 18
    7.7. Interface utilisateur 18
    7.8. Flux complet de fonctionnement de l'application 18
8. Conclusion 20

Remerciements

La réalisation de ce travail a été rendue possible grâce à l'accompagnement et au soutien de nombreuses personnes.

Les remerciements sont adressés à tous ceux qui ont contribué, directement ou indirectement, à l'accomplissement de ce projet :

— Aux enseignants et encadrants pour leurs conseils, leur expertise et leur soutien tout au long de ce travail.

— Aux collègues et camarades pour les échanges enrichissants et l'aide apportée durant le projet.

— À la famille et aux proches pour l'encouragement et le soutien moral constants.

L'ensemble de ces contributions a été précieux et a permis la réussite de ce travail.

1. Introduction

1.1. Énoncé du projet

Ce mini-projet s'inscrit dans le cadre du module de Génie Logiciel en 3 année Informatique. Il consiste à analyser, modéliser et développer une application de gestion des vols au sein d'une compagnie aérienne.

Dans une compagnie aérienne, plusieurs vols sont proposés. Chaque vol est assuré par un avion et un pilote. Une compagnie peut offrir plusieurs vols caractérisés par un numéro, une ville de départ et une ville d'arrivée. Un vol est obligatoirement associé à un avion, défini par un code, un type, un modèle et une capacité en nombre de passagers. De même, chaque vol est piloté par un seul pilote possédant un identifiant, un nom et un prénom.

Le projet demande également la modélisation UML (cas d'utilisation et diagramme de classes), la traduction du modèle en schéma relationnel, puis la création d'une base de données SQL et la réalisation d'une application permettant de gérer ces informations.

1.2. Objectifs du mini-projet

Les objectifs principaux du mini-projet sont les suivants :

Objectifs d'analyse et de conception

- Comprendre le fonctionnement global d'une compagnie aérienne en termes de gestion des vols.
- Identifier et formuler les besoins fonctionnels du système.
- Concevoir les diagrammes UML nécessaires :
  - Diagrammes de cas d'utilisation,
  - Diagramme de classes.
- Traduire le diagramme de classes en modèle relationnel cohérent.

Objectifs techniques

- Créer et implémenter la base de données MySQL correspondant au modèle relationnel.
- Développer une application permettant :
  - La gestion des compagnies, avions, pilotes et vols,
  - L'affectation d'un avion et d'un pilote à chaque vol,
  - La consultation des informations liées aux vols.

Objectifs pédagogiques

- Appliquer les principes de la modélisation orientée objet.
- Manipuler les outils UML tels que StarUML.
- Mettre en pratique des compétences en SQL (création, insertion, gestion des relations).
- Développer une application fonctionnelle et structurée.
- Produire un rapport clair et professionnel expliquant toutes les étapes du projet.

2. Cahier des charges

2.1. Présentation générale

Contexte. Une compagnie aérienne souhaite mettre en place un système informatique de gestion regroupant l'ensemble des informations relatives aux compagnies, aux avions, aux pilotes ainsi qu'aux vols. L'objectif principal est d'assurer une gestion fiable, cohérente et sécurisée des opérations aériennes.

Objectifs. Le système devra permettre :
- la gestion des avions et de leurs affectations aux vols ;
- la gestion des pilotes ;
- la planification, la consultation et le suivi des vols ;
- la garantie de l'intégrité des données (aucun vol sans avion, pilote ou compagnie).

2.2. Périmètre du système

Fonctionnalités prises en charge :
- enregistrement, modification et suppression des avions et pilotes ;
- création, modification et consultation des vols ;
- recherche et filtrage des vols (par numéro, avion ou pilote).

Fonctionnalités exclues :
- réservation ou vente de billets ;
- suivi en temps réel des vols ;
- gestion de maintenance ou réparations des avions.

2.3. Exigences fonctionnelles

Gestion des avions

Chaque avion possède les informations suivantes : code, type, modèle, nombre de places et compagnie propriétaire. Fonctionnalités :
- ajouter, modifier et supprimer un avion ;
- consulter les avions appartenant à une compagnie donnée ;
- affecter un avion à différents vols dans le temps.

Gestion des pilotes

Chaque pilote est décrit par : identifiant, nom et prénom. Fonctionnalités :
- ajouter, modifier et supprimer un pilote ;
- consulter la liste des pilotes enregistrés.

Gestion des vols

Un vol est caractérisé par : numéro, ville de départ, ville d'arrivée, date et heure, nombre de passagers, avion affecté, pilote affecté et compagnie. Fonctionnalités :
- créer, modifier et supprimer un vol ;
- rechercher et consulter les vols ;
- garantir automatiquement :
  - un seul avion par vol ;

— un seul pilote par vol ;
— une seule compagnie par vol.

2.4. Contraintes fonctionnelles

— un vol est associé à une seule compagnie aérienne ;
— un avion peut être assigné à plusieurs vols au cours du temps ;
— un pilote peut piloter plusieurs vols ;
— chaque vol doit obligatoirement spécifier une ville de départ et d'arrivée ;
— l'intégrité des données doit être garantie par des clés étrangères (avion, pilote, compagnie).

2.5. Exigences non fonctionnelles

Performance. Les opérations de consultation doivent répondre en moins de 2 secondes.

Fiabilité. Le système doit afficher des messages d'erreur clairs, confirmer les suppressions et journaliser les actions importantes.

Sécurité. Accès au système par authentification sécurisée ; audit des opérations sensibles.

Ergonomie. Interface intuitive avec navigation fluide entre la gestion des compagnies, des avions, des pilotes et des vols.

Compatibilité. Application web compatible avec les navigateurs modernes ; base de données SQL (Oracle, MySQL ou PostgreSQL).

2.6. Modèle conceptuel

— **Compagnie** : CodeCompagnie, Nom, SiegeSocial
— **Avion** : CodeAvion, Type, Modele, NbPlaces, CodeCompagnie
— **Pilote** : IdPilote, Nom, Prenom
— **Vol** : NumVol, Depart, Arrivee, DateHeure, NbPassagers, CodeAvion, IdPilote, CodeCompagnie

2.7. Livrables

— cahier des charges ;
— diagramme de cas d'utilisation ;
— diagramme de classes UML ;
— modèle relationnel ;
— script SQL complet.

2.8. Technologies recommandées

— Backend : Java ;
— Base de données : MySQL ;
— Outils de modélisation : StarUML.

3. Fonctionnalités du système

Le système doit offrir un ensemble complet de fonctionnalités permettant la gestion efficace des compagnies aériennes, des avions, des pilotes ainsi que des vols. Ces fonctionnalités sont organisées en modules pour plus de clarté.

3.1. Gestion des avions

— **Ajouter un avion** Informations : code avion, type, modèle, capacité passagers. Vérification : le code avion doit être unique.
— **Modifier un avion** Modification : type, modèle, capacité passagers.
— **Supprimer un avion** Refuser la suppression si l'avion est utilisé dans des vols actifs.
— **Consulter la liste des avions** Affichage complet de tous les détails des avions.
— **Afficher les vols assurés par un avion** Liste de tous les vols dans lesquels l'avion a été utilisé.

3.2. Gestion des pilotes

— **Ajouter un pilote** Informations : identifiant, nom, prénom.
— **Modifier un pilote** Modification : nom et prénom.
— **Supprimer un pilote** Interdire la suppression si le pilote est affecté à des vols.
— **Consulter la liste des pilotes** Affichage de tous les pilotes enregistrés.
— **Voir les vols effectués par un pilote** Liste des vols associés à ce pilote.

3.3. Gestion des vols

— **Ajouter un vol** Informations : numéro de vol (unique), ville de départ, ville d'arrivée, date du vol, compagnie, avion, pilote.
— **Modifier un vol** Modification de toutes les informations sauf le numéro du vol.
— **Supprimer un vol** Suppression autorisée si le vol n'est plus nécessaire.
— **Affecter un avion à un vol** Sélection d'un avion disponible.
— **Affecter un pilote à un vol** Sélection d'un pilote disponible.
— **Consulter la liste des vols** Affichage : numéro, avion, pilote, villes de départ et d'arrivée, date, compagnie.

3.4. Fonctionnalités de recherche

— Recherche par numéro de vol (exact).
— Recherche par avion (ID/code) → liste des vols assurés.
— Recherche par pilote → vols pilotés.
— Recherche par date → vols programmés ce jour.

— Recherche par lieu de départ → liste correspondante.
— Recherche par lieu d'arrivée → liste correspondante.
— Recherche multi-critères (ex. avion + date, pilote + ville de départ).

3.5. Fonctionnalités générales du système

— **Gestion des erreurs et validations** Vérification des champs obligatoires. Messages clairs : « pilote indisponible », « vol déjà existant », etc.
— **Interface conviviale** Menus : Compagnie / Avion / Pilote / Vol / Recherche. Navigation intuitive.
— **Intégrité des données** Aucune suppression ne doit créer d'incohérence. Utilisation systématique des clés primaires et étrangères.

3.6. Contraintes fonctionnelles

— Un vol est associé à une seule compagnie, un seul avion et un seul pilote.
— Un avion peut assurer plusieurs vols dans le temps.
— Un pilote peut piloter plusieurs vols.
— Les villes de départ et d'arrivée doivent être renseignées pour chaque vol.
— Le système doit garantir l'intégrité : pas de vol sans pilote ni avion ; aucune suppression incohérente.

4. Diagrammes UML

4.1 Diagramme de cas d'utilisation

Le diagramme de cas d'utilisation ci-dessous présente une vue globale des interactions entre l'utilisateur et le système de gestion des vols. Il illustre les principales fonctionnalités accessibles ainsi que les différents acteurs impliqués dans le processus de gestion.

Ce diagramme permet de comprendre rapidement les attentes fonctionnelles du système, en mettant en évidence :

— **L'authentification de l'utilisateur** : point d'entrée obligatoire pour garantir la sécurité du système.
— **La gestion des entités principales** : compagnies, avions, pilotes et vols. Pour chacune de ces entités, l'utilisateur peut effectuer les opérations CRUD (Créer, Lire, Mettre à jour, Supprimer).
— **Les fonctionnalités de recherche avancée** : le système offre la possibilité de rechercher un vol par numéro, par avion, par pilote, par date ou par ville (départ/arrivée).
— **L'intégrité des données** : par exemple, un vol ne peut être créé sans associer une compagnie, un avion et un pilote.

Le diagramme permet ainsi de visualiser les rôles du système et ses frontières fonctionnelles, facilitant la compréhension générale du projet.

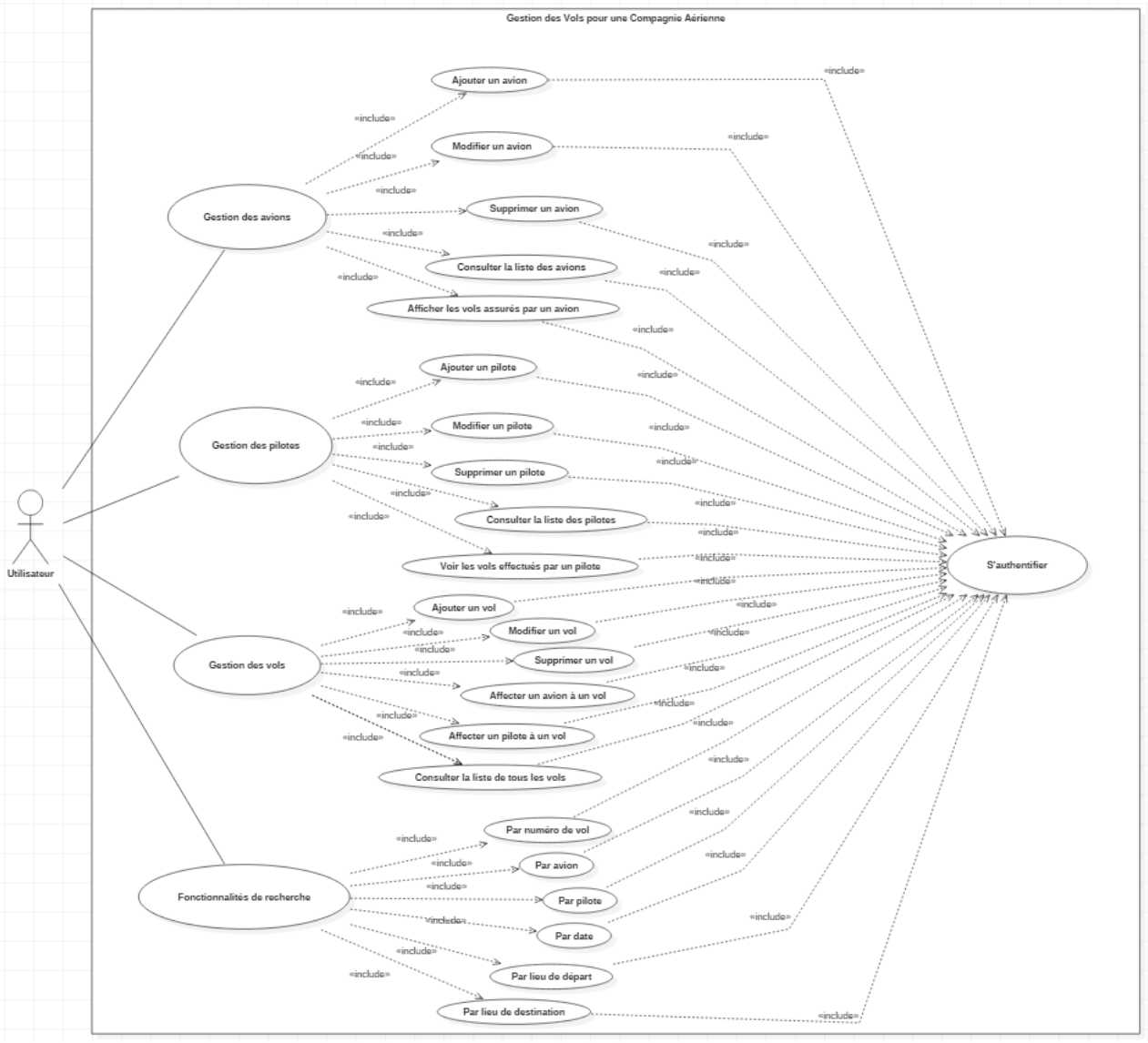


FIGURE 1 – Diagramme de cas d'utilisation du système de gestion des vols

4.2 Diagramme de classes

Le diagramme de classes présenté ci-dessous décrit la structure statique du système de gestion des vols. Il met en évidence les différentes classes ainsi que les relations qui les unissent (associations, compositions, dépendances).

Ce diagramme permet d'identifier clairement :

— **Les classes principales** : Compagnie, Avion, Pilote et Vol.

— **Leurs attributs essentiels** : par exemple, un avion possède un identifiant, une capacité et un modèle ; un vol possède un numéro, une date, une ville de départ et d'arrivée, etc.

— **Les relations entre les classes** :

  — Une **compagnie** possède plusieurs avions et plusieurs pilotes.

  — Un **avion** peut être associé à plusieurs vols (au cours du temps).

  — Un **pilote** peut piloter plusieurs vols.

  — Un **vol** est obligatoirement associé à une compagnie, un avion et un pilote.

— **Les multiplicités** : elles définissent les règles métiers, comme la gestion des vols ou l'affectation des pilotes.

Ce diagramme constitue une base solide pour la conception orientée objet, facilitant la transition vers l'implémentation.

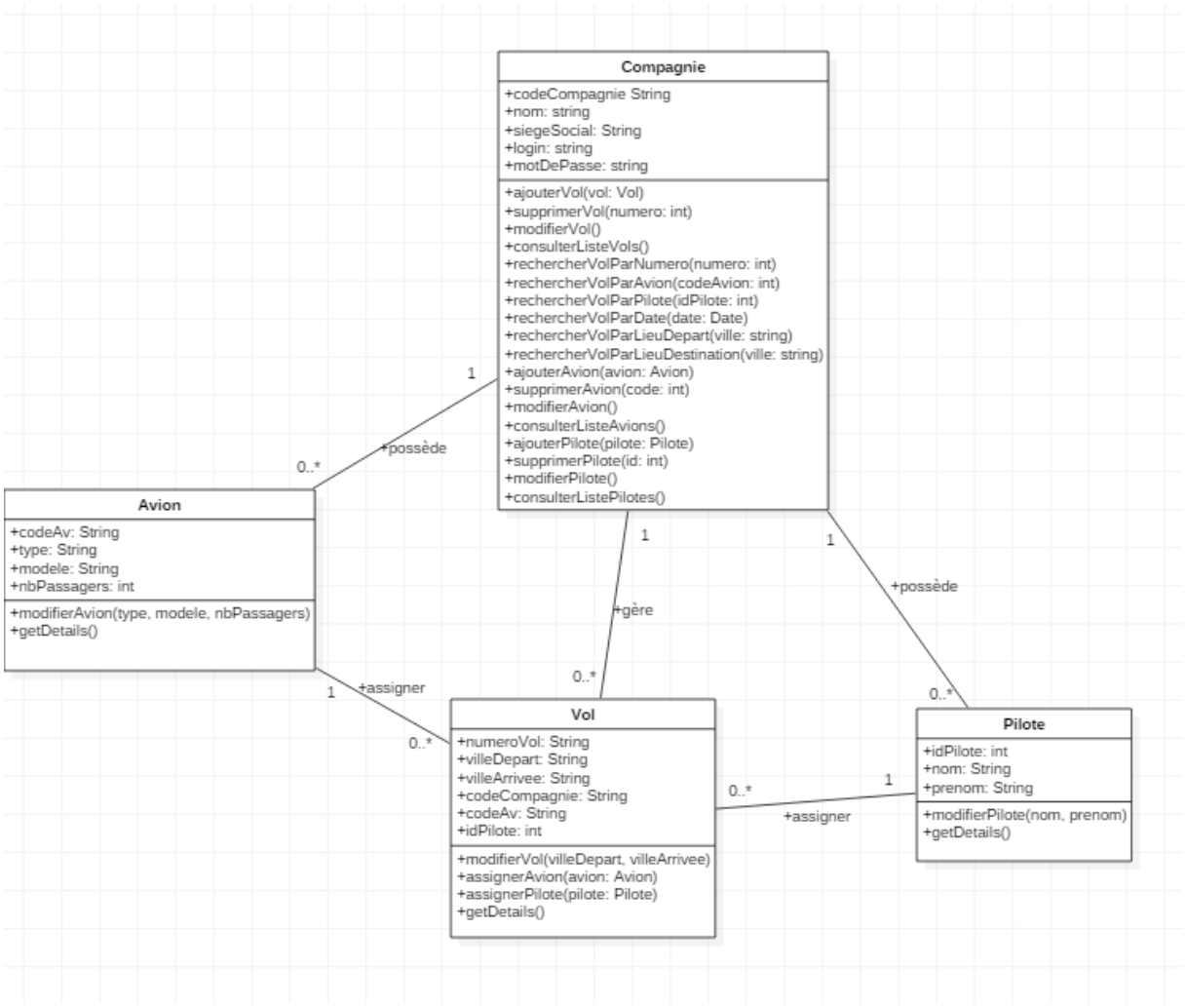


FIGURE 2 – Diagramme de classes du système de gestion des vols

4.3 Diagrammes de séquence

Les diagrammes de séquence présentés dans cette section illustrent les interactions entre les acteurs et le système pour trois processus essentiels : la gestion des avions, la gestion des pilotes et la gestion des vols. Chaque diagramme met en évidence la chronologie des échanges entre les différentes entités du système, permettant de comprendre la dynamique opérationnelle propre à chaque fonctionnalité.

Diagramme de séquence : Gestion des avions

Ce diagramme illustre le processus de gestion des avions, incluant l'ajout, la modification, la suppression et la vérification de disponibilité des appareils au sein du système.

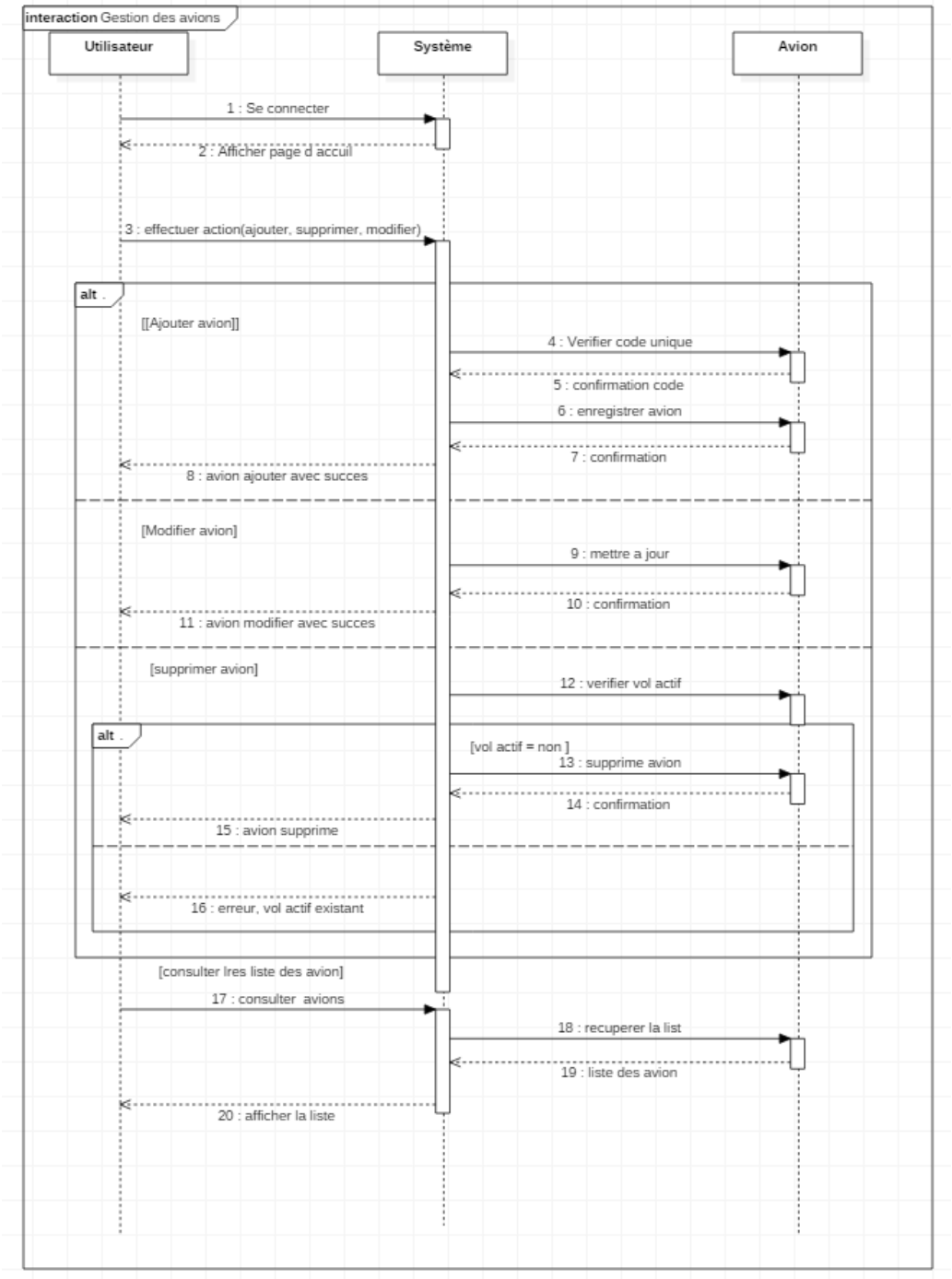


FIGURE 3 – Diagramme de séquence pour la gestion des avions

Diagramme de séquence : Gestion des pilotes

Ce diagramme présente les interactions liées à l'ajout, la mise à jour, la planification et la vérification de disponibilité des pilotes dans le système.

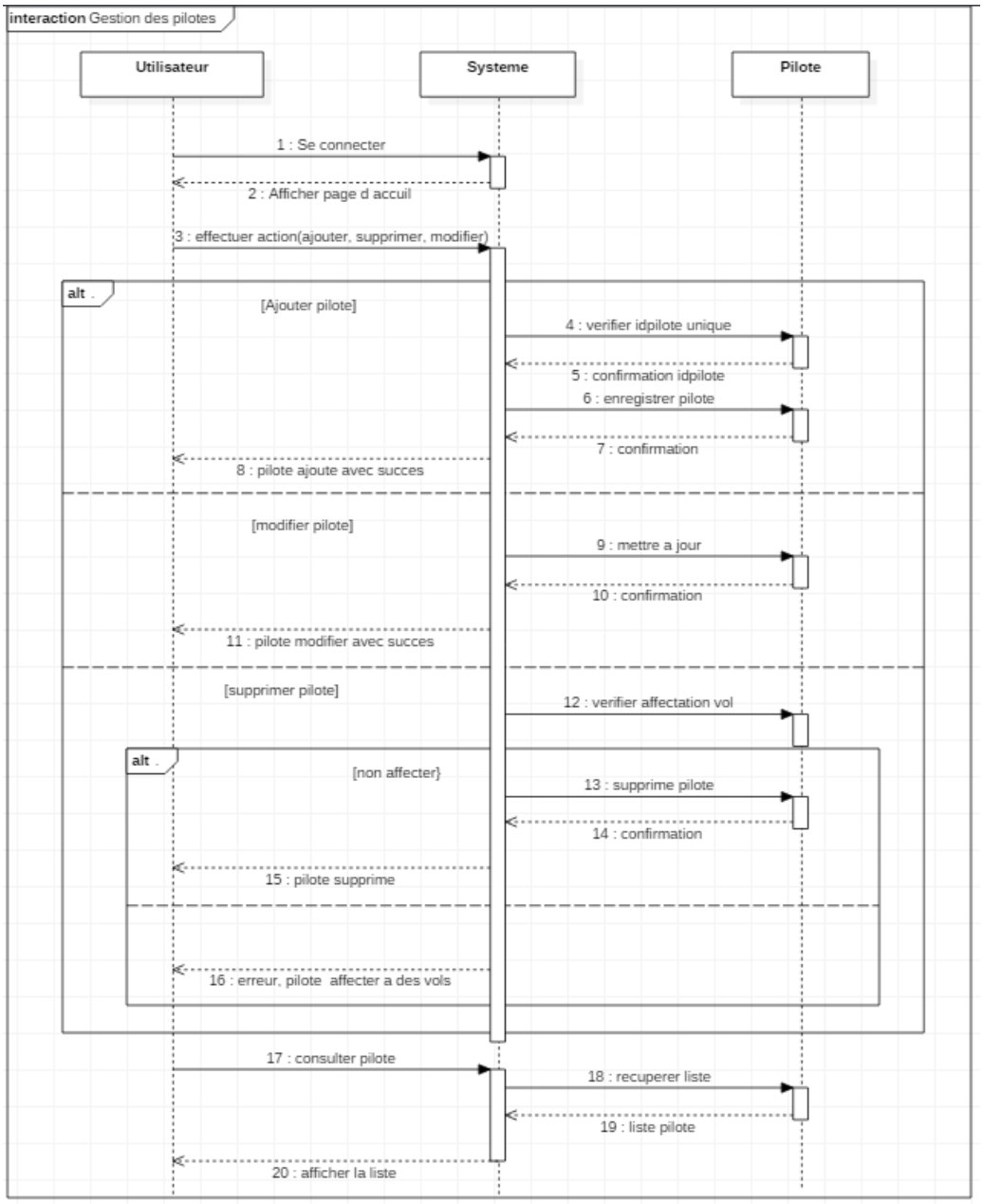


FIGURE 4 – Diagramme de séquence pour la gestion des pilotes

Diagramme de séquence : Gestion des vols

Ce diagramme décrit les interactions permettant de créer un vol, d'affecter un pilote et un avion, de vérifier les conflits d'horaires et de valider la planification finale.

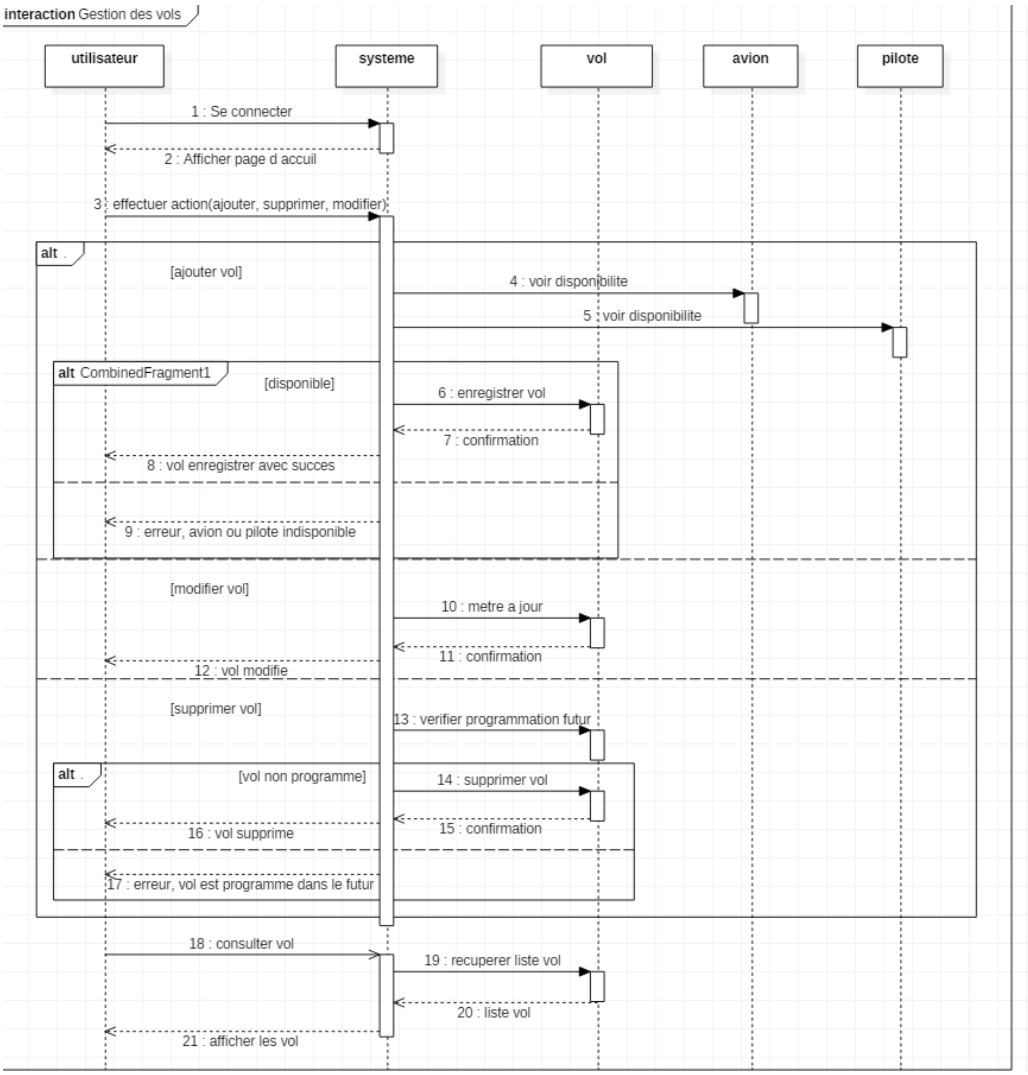


FIGURE 5 – Diagramme de séquence pour la gestion des vols

5. Modèle de données (SQL)

Le modèle de données ci-dessous représente la structure relationnelle de la base de données du système de gestion des vols. Il définit les tables principales, leurs attributs, les contraintes d'intégrité, ainsi que les relations entre elles.

Ce modèle permet de :

— Définir les entités principales : **Compagnie**, **Avion**, **Pilote**, **Vol**.

— Garantir l'intégrité des données grâce aux clés primaires, clés étrangères et contraintes de vérification.

— Illustrer les relations entre les entités, notamment l'appartenance des avions et des pilotes à une compagnie, et l'association des vols avec une compagnie, un avion et un pilote.

5.1 Script SQL

```
-- ================================
-- 1. TABLE Compagnie
-- ================================
CREATE TABLE Compagnie (
    codeCompagnie VARCHAR(10) PRIMARY KEY,
    nom           VARCHAR(100) NOT NULL,
    siegeSocial   VARCHAR(150),
    login         VARCHAR(50) UNIQUE NOT NULL,
    motDePasse    VARCHAR(100) NOT NULL
);

-- ================================
-- 2. TABLE Avion
-- ================================
CREATE TABLE Avion (
    codeAv      VARCHAR(10) PRIMARY KEY,
    type        VARCHAR(50) NOT NULL,
    modele      VARCHAR(50) NOT NULL,
    nbPassagers INT CHECK(nbPassagers > 0),
    codeCompagnie VARCHAR(10) NOT NULL,

    CONSTRAINT fk_Avion_Compagnie
        FOREIGN KEY (codeCompagnie)
        REFERENCES Compagnie(codeCompagnie)
        ON DELETE CASCADE
);

-- ================================
-- 3. TABLE Pilote
-- ================================
CREATE TABLE Pilote (
    idPilote  INT PRIMARY KEY AUTO_INCREMENT,
    nom       VARCHAR(100) NOT NULL,
```

```
    prenom    VARCHAR(100) NOT NULL,
    codeCompagnie VARCHAR(10) NOT NULL,

    CONSTRAINT fk_Pilote_Compagnie
        FOREIGN KEY (codeCompagnie)
        REFERENCES Compagnie(codeCompagnie)
        ON DELETE CASCADE
);

-- ================================
-- 4. TABLE Vol
-- ================================
CREATE TABLE Vol (
    numeroVol     VARCHAR(10) PRIMARY KEY,
    villeDepart   VARCHAR(100) NOT NULL,
    villeArrivee  VARCHAR(100) NOT NULL,
    dateVol       DATE NOT NULL,

    codeCompagnie VARCHAR(10) NOT NULL,
    codeAv        VARCHAR(10) NOT NULL,
    idPilote      INT NOT NULL,

    CONSTRAINT fk_Vol_Compagnie
        FOREIGN KEY (codeCompagnie)
        REFERENCES Compagnie(codeCompagnie)
        ON DELETE CASCADE,

    CONSTRAINT fk_Vol_Avion
        FOREIGN KEY (codeAv)
        REFERENCES Avion(codeAv)
        ON DELETE RESTRICT,

    CONSTRAINT fk_Vol_Pilote
        FOREIGN KEY (idPilote)
        REFERENCES Pilote(idPilote)
        ON DELETE RESTRICT
);
```

6. Modèle relationnel simplifié

Cette section présente un modèle relationnel simplifié du système de gestion des vols. Il met en évidence les principales entités ainsi que leurs relations, permettant de comprendre rapidement l'organisation des données.

6.1. Tables principales

- **Compagnie** : identifiée par `codeCompagnie`. Contient : nom, siège social.
- **Avion** : identifié par `codeAv`. Contient : type, modèle, nombre de passagers, et référence la compagnie propriétaire (`codeCompagnie`).
- **Pilote** : identifié par `idPilote`. Contient : nom, prénom, et référence la compagnie d'affiliation (`codeCompagnie`).
- **Vol** : identifié par `numeroVol`. Contient : ville de départ, ville d'arrivée, date, et références vers la compagnie (`codeCompagnie`), l'avion (`codeAv`) et le pilote (`idPilote`) responsables du vol.

6.2. Relations entre les tables

- Une **compagnie** peut posséder plusieurs **avions** et plusieurs **pilotes** (relation 1..*).
- Un **avion** peut effectuer plusieurs **vols** (relation 0..*).
- Un **pilote** peut piloter plusieurs **vols** (relation 0..*).
- Chaque **vol** est associé à exactement une **compagnie**, un **avion** et un **pilote** (relation 1..1 pour chaque association).

6.3. Diagramme relationnel simplifié (textuel)

```
Compagnie (codeCompagnie)
    |
    |---< Avion (codeAv, codeCompagnie)
    |
    |---< Pilote (idPilote, codeCompagnie)
    |
    |---< Vol (numeroVol, codeCompagnie, codeAv, idPilote)
```

Remarques :

- Le modèle met en avant uniquement les clés principales et les relations essentielles.
- Il est volontairement simplifié pour être facilement compréhensible.
- Les détails sur les contraintes d'intégrité ou les vérifications supplémentaires (comme `ON DELETE`) sont omis pour plus de clarté.

7. Fonctionnement détaillé de l'application

Cette section décrit de manière détaillée le fonctionnement de l'application de gestion des vols, en expliquant les différents modules, les interactions entre l'utilisateur et le système, ainsi que le traitement des données. L'objectif est de montrer comment chaque fonctionnalité est exécutée et comment l'intégrité des données est garantie.

7.1. Authentification et accès au système

- L'utilisateur accède à l'application via une page de connexion sécurisée.
- Les identifiants (*login* et *mot de passe*) sont vérifiés dans la table `Compagnie`.
- En cas de succès, l'utilisateur est redirigé vers le tableau de bord principal où il peut accéder aux modules de gestion.
- En cas d'erreur (login ou mot de passe incorrect), un message clair s'affiche : *"Identifiants incorrects, veuillez réessayer"*.
- Exemple : si la compagnie *AirAlgérie* tente de se connecter avec un login inexistant, l'accès est refusé et l'utilisateur doit réessayer.

7.2. Gestion des avions

- **Création d'un avion** : l'utilisateur saisit le code avion, le type (ex : Boeing 737), le modèle et la capacité en passagers. L'avion est automatiquement associé à la compagnie de l'utilisateur.
- **Modification** : l'utilisateur peut mettre à jour le type, le modèle ou la capacité. Le code avion reste unique et non modifiable.
- **Suppression** : un avion ne peut être supprimé que s'il n'est pas affecté à des vols actifs. Sinon, un message d'erreur s'affiche.
- **Consultation** : l'utilisateur peut afficher tous les avions, filtrer par compagnie ou rechercher un avion spécifique par code.
- **Exemple pratique** : l'avion "BO737-123" affecté à trois vols existants ne peut pas être supprimé. L'utilisateur devra d'abord réaffecter ou supprimer les vols correspondants.

7.3. Gestion des pilotes

- **Création** : ajout d'un pilote avec identifiant unique, nom et prénom.
- **Modification** : possibilité de mettre à jour le nom et le prénom.
- **Suppression** : refusée si le pilote est affecté à un vol existant.
- **Consultation** : l'utilisateur peut voir la liste complète des pilotes, filtrer par compagnie ou consulter les vols pilotés.
- **Exemple** : le pilote "Ali Ben Ahmed" affecté au vol "AA123" ne peut pas être supprimé sans réassignation du vol.

7.4. Gestion des vols

- **Création** : l'utilisateur saisit le numéro de vol, la ville de départ et d'arrivée, la date, l'avion et le pilote disponibles, ainsi que la compagnie. Le système vérifie automatiquement la disponibilité de l'avion et du pilote pour éviter les conflits.

— **Modification** : toutes les informations peuvent être mises à jour sauf le numéro du vol, qui reste unique.
— **Suppression** : autorisée uniquement si le vol n'est plus nécessaire ou programmé dans le futur.
— **Consultation** : affichage complet avec possibilité de filtrer par date, avion, pilote ou ville.
— **Exemple** : pour créer le vol "AA456" de "Alger" vers "Paris" le 15/12/2025, le système vérifie que l'avion sélectionné n'est pas déjà affecté à un autre vol le même jour et que le pilote est disponible.

7.5. Recherche et filtrage avancés

— Recherche par numéro de vol, par avion ou par pilote.
— Filtrage par date, ville de départ ou d'arrivée.
— Recherche multi-critères : par exemple, vols d'un avion donné entre deux dates spécifiques.
— Les résultats sont affichés de manière synthétique et détaillée, avec toutes les informations nécessaires.

7.6. Gestion des erreurs et intégrité des données

— Vérification systématique des champs obligatoires avant validation.
— Messages d'erreur clairs et précis, par exemple :
  — "Pilote indisponible à cette date"
  — "Vol déjà existant"
  — "Avion déjà affecté à un autre vol"
— Les contraintes relationnelles (PK, FK) dans la base SQL garantissent que toutes les opérations respectent l'intégrité des données.

7.7. Interface utilisateur

— Menu principal avec accès direct aux modules : Compagnie, Avion, Pilote, Vol et Recherche.
— Tableaux interactifs pour chaque entité avec options de tri, filtrage et recherche.
— Formulaires clairs pour la saisie et la modification des données.
— Messages et confirmations lors des actions critiques (création, suppression, modification).

7.8. Flux complet de fonctionnement de l'application

1. L'utilisateur se connecte au système via l'interface sécurisée.
2. Il choisit le module approprié selon l'action à effectuer : Compagnie, Avion, Pilote ou Vol.
3. L'utilisateur effectue l'opération souhaitée : création, modification, suppression ou consultation.
4. Pour la gestion des vols, le système vérifie la disponibilité de l'avion et du pilote pour éviter tout conflit.

5. Les données validées sont enregistrées dans la base SQL, respectant toutes les contraintes de clés primaires et étrangères.
6. L'utilisateur peut effectuer des recherches ou générer des listes filtrées en fonction de critères précis.
7. Après chaque action, l'application affiche un message de confirmation ou d'erreur.

Remarque : Ce fonctionnement détaillé permet de gérer l'ensemble du système de manière sécurisée, cohérente et efficace, tout en assurant la disponibilité et l'intégrité des informations à tout moment.

8. Conclusion

Ce mini-projet a permis de concevoir et de développer une application complète de gestion des vols pour une compagnie aérienne. À travers l'analyse des besoins, la modélisation UML, la traduction en modèle relationnel et la mise en œuvre de la base de données SQL, nous avons pu créer un système structuré et fonctionnel répondant aux exigences définies dans le cahier des charges.

Les points clés de ce projet sont les suivants :

— Une compréhension approfondie des processus de gestion d'une compagnie aérienne, incluant la gestion des avions, des pilotes et des vols.
— La conception de diagrammes UML (cas d'utilisation, classes et séquence) permettant de formaliser les besoins et de faciliter la conception orientée objet.
— La traduction du modèle conceptuel en un modèle relationnel clair, garantissant l'intégrité et la cohérence des données.
— Le développement d'une base de données SQL complète, ainsi qu'une application intuitive et sécurisée pour gérer toutes les entités.
— L'application des bonnes pratiques en génie logiciel, en conception orientée objet, en développement d'interface utilisateur et en gestion des données relationnelles.

Perspectives d'amélioration :

— Intégration d'un module de réservation et de suivi en temps réel des vols.
— Mise en place d'un tableau de bord analytique pour suivre les performances de la compagnie et l'utilisation des avions et pilotes.
— Extension de l'application pour gérer la maintenance des avions et les alertes de sécurité.